Power Spectral Density of Line Code

Super-heterodyne Receiver

Youssef Mohammed Ibrahim Sayed

Youssef Hisham Ahmed Hikel

Mina Wael Tanagho Fahmy

Nader Hany Maurice Wahib

Abdelrahaman Ahmed Kamal Atta

Faculty of Engineering

Department of Electronics and Communications

Course code : EECG232-17

Course name : Introduction to Communication Systems

Power Spectral Density of Line Code

Super-heterodyne Receiver

يوسف محمد ابراهيم سيد

Student ID : 91240892
Section : 4
BN : 34

يوسف هشام أحمد هيكل

Student ID : 91240907
Section : 4
BN : 36

مينا وائل تناغو فهمي

Student ID : XXXXXXXX
Section : 4
BN : XX

نادر هاني موريس وهيب

Student ID : XXXXXXXX
Section : 4
BN : XX

عبدالرحمن أحمد كمال عطا

Student ID : XXXXXXXX
Section : 4
BN : XX

Supervisor: Dr. Mohammed Nafea
Professor, Cairo University

Co-supervisor: ENG. Mohammed Khaled
Teacher Assistant, Cairo University

Faculty of Engineering

Department of Electronics and Communications

Course code : EECG232-17

Course name : Introduction to Communication Systems

Abstract

Contents

1 Problem Description 2

2 Introduction 3

3 Control Flags 4

4 Generation of Data 5

5 Creating the Unipolar Ensemble 6

6 Creating The Polar RZ Ensemble 7

7 Applying Random Initial Time Shifts 8

8 Preparing Cell Arrays For Analysis 9

9 Q1 Statistical Mean Calculation 10

10 Q2 Is the Process Stationary 11

11 Q3 Statistical Autocorrelation 12

12 Q4 Time Mean & Autocorrelation of One Waveform 13

13 Q5 Is the Random Process Ergodic 14

14 Plotting the PSD of the Ensemble 15

15 Q6 Signal Bandwidth 16

Appendices

A Appendix 18

Chapter 1

Problem Description

Chapter 2

Introduction

Chapter 3

Control Flags

Chapter 4

Generation of Data

Chapter 5

Creating the Unipolar Ensemble

Chapter 6

Creating The Polar RZ Ensemble

Chapter 7

Applying Random Initial Time Shifts

Chapter 8

Preparing Cell Arrays For Analysis

Chapter 9

Q1 Statistical Mean Calculation

Chapter 10

Q2 Is the Process Stationary

Chapter 11

Q3 Statistical Autocorrelation

Chapter 12

Q4 Time Mean & Autocorrelation of One Waveform

Chapter 13

Q5 Is the Random Process Ergodic

Chapter 14

Plotting the PSD of the Ensemble

Chapter 15

Q6 Signal Bandwidth

Appendices

Appendix A

Appendix

,

Listing 1: *Main Script — `AnalogCommProject.m`*

```
% main.m
% Authors : Youssef Hisham Ahmed, Youssef Mohammed Ibrahim
% Date : 4/4/2026
% Subject : Modulation schemes for various signals

clear; clc; close all;

%% Run Preprocessing and Modulator
preprocessing;
modulator;

%% Shared Filters Setup (IF & LPF)
F_IF = 15e3;

% IF Filtering Specs
IF_Filter_Order = 8;
IF_PassBand_Low = 5e3;
IF_PassBand_High = 30e3;
IF_Filter_Specs = fdesign.bandpass('N,F3dB1,F3dB2', IF_Filter_Order, IF_PassBand_Low,
↪  IF_PassBand_High, Fs1);
IF_filter = design(IF_Filter_Specs, 'butter');

% LPF Specs
LPF_Filter_Order = 8;
LPF_Cutoff = 10e3; % 10 kHz cutoff
LPF_Specs = fdesign.lowpass('N,F3dB', LPF_Filter_Order, LPF_Cutoff, Fs1);
LPF_filter = design(LPF_Specs, 'butter');

%% Channel Specific RF Filters & Oscillators
RF_Filter_Order = 26;

% Quran Channel RF Filter (100 kHz Center)
RF_PassBand_Low_Quran = 85e3;
RF_PassBand_High_Quran = 115e3;
```

```
RF_Filter_Specs_Quran = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order, RF_PassBand_Low_Quran, RF_PassBand_High_Quran, Fs1);
RF_filter_Quran = design(RF_Filter_Specs_Quran, 'butter');
RF_output_Quran = filter(RF_filter_Quran, FDM);
F_osc_Quran = fc1 + F_IF;

% BBC Channel RF Filter (130 kHz Center)
RF_PassBand_Low_BBC = 115e3;
RF_PassBand_High_BBC = 145e3;
RF_Filter_Specs_BBC = fdesign.bandpass('N,F3dB1,F3dB2', RF_Filter_Order, RF_PassBand_Low_BBC, RF_PassBand_High_BBC, Fs1);
RF_filter_BBC = design(RF_Filter_Specs_BBC, 'butter');
RF_output_BBC = filter(RF_filter_BBC, FDM);
F_osc_BBC = fc2 + F_IF;

%% DEMODULATION

% Quran
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Ideal';
receiver;

% BBC
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Ideal';
receiver;

%% Q4 - NO RF Filter

% Quran Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_Quran;
station_name = 'Quran Channel';
current_condition = 'Q4 (No BPF)';
receiver;

% BBC Q4
current_RF_in = FDM; % Using raw FDM instead of filtered RF
current_F_osc = F_osc_BBC;
station_name = 'BBC Channel';
current_condition = 'Q4 (No BPF)';
receiver;

%% Q5 - MIXER OFFSET (0.1 kHz)

% Quran Q5 (0.1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 0.1e3;
```

```
station_name = 'Quran Channel';
current_condition = 'Q5 (0.1kHz Offset)';
receiver;

% BBC Q5 (0.1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 0.1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (0.1kHz Offset)';
receiver;

%% Q5 - MIXER OFFSET (1 kHz)

% Quran Q5 (1kHz)
current_RF_in = RF_output_Quran;
current_F_osc = F_osc_Quran + 1e3;
station_name = 'Quran Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;

% BBC Q5 (1kHz)
current_RF_in = RF_output_BBC;
current_F_osc = F_osc_BBC + 1e3;
station_name = 'BBC Channel';
current_condition = 'Q5 (1kHz Offset)';
receiver;

disp('All processing complete!');
export_figures;
```

Listing 2: *Signal Pre-processing — `preprocessing.m`*

```
% preprocessing.m

%% Reading the files
[signal1_raw, Fs1] = audioread("Short_QuranPalestine.wav");
[signal2_raw, Fs2] = audioread("Short_BBCArabic2.wav");

%% Convert stereo to mono
if size(signal1_raw, 2) == 2, signal1 = signal1_raw(:,1) + signal1_raw(:,2); else,
↪  signal1 = signal1_raw; end
if size(signal2_raw, 2) == 2, signal2 = signal2_raw(:,1) + signal2_raw(:,2); else,
↪  signal2 = signal2_raw; end

disp(['Fs1 = ', num2str(Fs1), ' Hz']);
disp(['Fs2 = ', num2str(Fs2), ' Hz']);

%% Pad to equal length
len1 = length(signal1);
len2 = length(signal2);
max_len = max(len1, len2);
```

```
if len1 < max_len, signal1 = [signal1; zeros(max_len - len1, 1)]; end
if len2 < max_len, signal2 = [signal2; zeros(max_len - len2, 1)]; end

%% Plot Spectrum (Original Baseband)
N_plot1 = length(signal1);
Y1_shifted = fftshift(fft(signal1));
Y2_shifted = fftshift(fft(signal2));

magnitude1_shifted = abs(Y1_shifted);
magnitude2_shifted = abs(Y2_shifted);
freq_shifted = (-N_plot1/2 : N_plot1/2 - 1) * (Fs1 / N_plot1);

figure('Name', 'Original Baseband Spectra');
subplot(2,1,1);
plot(freq_shifted / 1000, magnitude1_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('Spectrum of Signal 1 (Quran Channel)'); grid on; xlim([-25, 25]);

subplot(2,1,2);
plot(freq_shifted / 1000, magnitude2_shifted);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('Spectrum of Signal 2 (BBC Channel)'); grid on; xlim([-25, 25]);
drawnow;

%% Upsampling
upsample_factor = 10;
signal1 = interp(signal1, upsample_factor);
signal2 = interp(signal2, upsample_factor);

original_Fs = Fs1;
Fs1 = Fs1 * upsample_factor;
Fs2 = Fs2 * upsample_factor;
disp(['New Fs1 = ', num2str(Fs1), ' Hz']);
disp(['New Fs2 = ', num2str(Fs2), ' Hz']);
```

Listing 3: *AM-DSB-SC Modulator — `modulator.m`*

```
% modulator.m
disp('--- Running Modulator ---');

%% MODULATION
N1 = length(signal1);
Ts = 1/Fs1;
t = (0:N1-1)' * Ts;

fc = 100e3;            % Base carrier frequency
delta_f = 30e3;        % Frequency spacing
fc1 = fc;              % Carrier for first signal (100 kHz)
fc2 = fc + delta_f;    % Carrier for second signal (130 kHz)

```

```
Quran_Channel_Modulated = signal1 .* cos(2 * pi * fc1 * t);
BBC_Channel_Modulated   = signal2 .* cos(2 * pi * fc2 * t);

FDM = Quran_Channel_Modulated + BBC_Channel_Modulated;

%% Plotting FDM
Y_FDM = fft(FDM);
Y_FDM_shifted = fftshift(Y_FDM);
magnitude_FDM = abs(Y_FDM_shifted);
freq_axis = (-N1/2 : N1/2 - 1) * (Fs1 / N1);
freq_plot = freq_axis;

figure('Name', 'FDM Spectrum');
plot(freq_axis / 1000, magnitude_FDM);
xlabel('Frequency (kHz)'); ylabel('Magnitude');
title('FDM Signal Spectrum'); grid on; xlim([-200, 200]);
hold on;
xline(fc1/1000, 'r--', '100 kHz');
xline(fc2/1000, 'g--', '130 kHz');
xline(-fc1/1000, 'r--', '-100 kHz');
xline(-fc2/1000, 'g--', '-130 kHz');
legend('FDM', 'Station 1 (Quran)', 'Station 2 (BBC)');
drawnow;
```

Listing 4: *Super-heterodyne Receiver Script — `receiver.m`*

```
% receiver.m

% Mixer Stage
Mixer_Output = current_RF_in .* cos(2 * pi * current_F_osc * t);

% IF Filtering Process
IF_Channel = filter(IF_filter, Mixer_Output);

% Return to Baseband & Low-Pass Filtering
baseband_mixed = IF_Channel .* cos(2 * pi * F_IF * t);
demodulated = filter(LPF_filter, baseband_mixed);
demodulated = demodulated - mean(demodulated); % Remove DC offset

% Downsample back & Normalize
demod_down = downsample(demodulated, upsample_factor);
demod_down = demod_down / max(abs(demod_down));

% Plotting
plot_spectra(current_RF_in, IF_Channel, demodulated, freq_plot, station_name,
↪  current_condition);

% Audio Playback
sound(demod_down, original_Fs);
pause((length(demod_down) / original_Fs) + 1);
```

Listing 5: *Spectrum Plotting Function — `plot_spectra.m`*

```
% plot_spectra.m
function plot_spectra(RF_sig, IF_sig, base_sig, freq_plot, channel_name, condition)

    figure('Name', sprintf('%s - %s', channel_name, condition));

    % RF Stage Plot
    subplot(3,1,1);
    plot(freq_plot / 1000, abs(fftshift(fft(RF_sig))));
    title(['RF Output: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-200, 200]);

    % IF Stage Plot
    subplot(3,1,2);
    plot(freq_plot / 1000, abs(fftshift(fft(IF_sig))));
    title(['IF Output: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-50, 50]);

    % Baseband Plot
    subplot(3,1,3);
    plot(freq_plot / 1000, abs(fftshift(fft(base_sig))));
    title(['Baseband: ', condition]);
    xlabel('Frequency (kHz)'); ylabel('Magnitude');
    grid on; xlim([-25, 25]);

    drawnow;
end
```